Chapter 5. Financial Data Governance

As financial markets expand, so do the methods and use cases for how financial data is collected, stored, and used. This has generated concerns within the financial industry as well as broader governing bodies that are pushing for solid data controls, quality assurance, privacy rules, and increased security measures. As a result, data governance frameworks have emerged as a promising approach to defining and implementing rules and principles for guiding data practices within financial institutions.

This chapter will provide a practical framework for financial data governance based on three key components: data quality, data integrity, and data security and privacy. First, I'll cover the basics of financial data governance. Then, I'll go into depth about each of the three components in the sections that follow.

Financial Data Governance

Data governance is critical to securing financial data, ensuring regulatory compliance, and fostering trust among stakeholders. By implementing robust data governance practices, financial institutions can safeguard sensitive information, adhere to legal requirements, and maintain the integrity of their financial operations.

Financial Data Governance Defined

Before defining what financial data governance is, let's examine a few existing definitions. For example:

> *Data governance is everything you do to ensure data is secure, private, accurate, available, and usable. It includes the actions people must take, the processes they must follow, and the technology that supports them throughout the data life cycle.*
>
> — Google Cloud

> *Data governance is, first and foremost, a data management function to ensure the quality, integrity, security, and usability of the data collected by an organization.*
>
> — Eryurek et al., *Data Governance: The Definitive Guide* (O'Reilly, 2021)

As you can see, data governance can be considered a data management function, a process, or simply a set of technological and cultural practices. Interestingly, all the above definitions share a common purpose, that of ensuring data quality, security, integrity, availability, and usability. Building on these ingredients, I define financial data governance as follows:

> *Financial data governance is a technical and cultural framework that establishes a set of rules, roles, practices, controls, and implementation guidelines to ensure the quality, integrity, security, and privacy of financial data in compliance with both general and financial domain–specific internal and external policies, standards, requirements, and regulations.*

There isn't a one-size-fits-all solution for financial data governance frameworks. Two financial institutions may follow the same set of principles to establish financial data governance; however, the final implementations are very likely to be unique to each institution. The reason has to do with the nature of the issues that different financial institutions might face in terms of data quality, security, privacy, integrity, and more. It also depends on the financial institution's internal organizational structure, culture, process standardization and harmonization, senior management support, and participation.

Financial Data Governance Justified

Defining and enforcing an effective financial data governance framework requires a nonnegligible investment. On the one hand, a concrete and functional data governance framework needs to be defined, implemented, and integrated within the financial institution's data infrastructure. On the other hand, employees and data users need to be trained and prepared to adhere to the established data governance principles in their daily work. As such, it is important to first understand the value proposition of a financial data governance framework for your institution.

Importantly, financial organizations are among those that require and benefit from data governance the most. This can be explained by two main factors: performance and risk management.

Financial data governance impacts performance in several ways. First, data governance drives high data quality standards, which is a major input to most financial operations and decisions. Brian Buzzelli attributes *operational inefficiency* (inefficient use of input to produce output) in the financial industry to poor data quality. It impacts financial institutions' ability to conduct business efficiently, gain insights into market activity, make informed investment decisions, respond on time to new events, and communicate accurate figures to stakeholders. Second, financial data governance saves employees the nuisance of constantly checking and rechecking data quality, integrity, privacy, and security. Third, with solid data governance principles in place, developers and business teams feel more confident in the quality and compliance of their applications and products.

Another critical reason for implementing data governance in financial institutions is to effectively manage risks associated with data, which can have significant implications for these institutions. Such risks include the following:

- Cyberattacks intended to steal data, damage organizational resources, or interfere with the operation of confidential systems
- Data breaches where sensitive data falls into the hands of unauthorized persons or organizations
- Discriminatory biases built into financial applications

- Erratic data injected in models that distorts the results
- Data loss due to lack of backups, snapshots, or archives
- Absence of firm-level risk oversight due to decentralized data processes
- Lack of visibility into the data processing steps
- Privacy risks when sharing data with third parties
- Impact on model prediction quality due to bad data
- Financial and reputational risks due to nonconformance with legal and regulatory requirements

Regulators around the world have put forth substantial efforts toward creating and enforcing laws and regulations that address the above risks. Some examples are listed here:

- Sarbanes–Oxley Act
- Bank Secrecy Act
- Basel Committee on Banking Supervision's standard number 239 (BCBS 239)
- European Union's (EU's) Solvency II Directive
- California Consumer Privacy Act (CCPA)
- EU's General Data Protection Regulation (GDPR)

In order to adhere to these regulations, financial institutions must now establish and implement robust data governance frameworks. Consequently, compliance has emerged as the primary driver for the adoption of data governance within the financial sector.

The topic of data governance is quite vast and can be complex to navigate. Practitioners, researchers, financial institutions, and consulting firms regularly publish data governance studies and guidelines. In this chapter, I will present a practical data governance framework centered on three major areas that are common to all financial institutions: data quality, data integrity, and data security and privacy.

Data Quality

Data quality measures how well a dataset satisfies its intended use in various operational and analytical applications. For financial institutions, data has a primary role as input in the decision-making and product development process. Consequently, the problem of financial data quality needs to be handled with care by financial data engineers, analysts, and machine learning experts. Some use the term *data downtime* to refer to periods during which data is not accessible or is unusable due to quality-related issues. In a data-driven financial institution, prolonged or frequent data downtimes can severely impact efficiency, erode customer trust, interrupt research endeavors, and influence management and investment decisions.

A *Data Quality Framework* (DQF) is required to ensure financial data quality. The definition and specifications of a DQF may vary from one institution to another based on internal and external factors.[1] In this chapter, I will share the main ingredients that you, as a financial data engineer, can leverage to define and build a DQF. Such ingredients are often called *data quality dimensions* (DQDs). A DQD refers to those attributes or

indicators of data quality that, if measured correctly, can convey information about the overall quality of financial data.

There isn't a fixed list of DQDs. As new requirements and data issues emerge, various DQDs can be identified and measured. Furthermore, the relevance of particular quality dimensions can vary depending on the specific problem being addressed and the needs of data consumers.[2] For example, certain aspects of data quality can have a greater influence on the performance of machine learning models.[3]

Therefore, educating your team on defining DQDs can greatly benefit your financial institution. To establish a baseline, in the following sections, I will present nine DQDs that are particularly relevant to financial data: errors, outliers, biases, granularity, duplicates, availability and completeness, timeliness, constraints, and relevance. Note that while the needs of your organization are unique and may vary, these DQDs are fairly universal and likely to apply to your business.

Dimension 1: Data Errors

Data errors are digital records that have been recorded erroneously, and therefore reflect invalid or incorrect values. The presence of data errors can compromise the value, accuracy, and reliability of the data and negatively impact reporting, analysis, and decision-making.

Data errors are quite common in financial data and represent the most frequent data quality issue for financial institutions. A global survey of over 1,100 financial executives and professionals conducted by BlackLine in 2018 revealed that 55% of respondents were not completely confident in their institution's ability to spot financial data errors before reporting results. The survey shows that 7 in 10 respondents believe that their institution made important decisions based on inaccurate or out-of-date financial data. The majority of C-level respondents agreed that there would be a negative impact if financial inaccuracies were not detected before reporting. The negative impacts included harm to the company's image, trouble obtaining new investments, rising debt levels, penalties, and even jail time.

There isn't a fixed list of financial data error types; rather, they materialize with the introduction of data sources, products, pipelines, and various manipulation and transformation operations. But to give a few examples, financial data errors can involve the following issues:

- Random measurement errors (e.g., $9.345 instead of $9.335)
- Wrong decimal places (e.g., a price of $111.34 instead off $11.134)
- Decimal precision (e.g., an exchange rate of 1.345 instead of 1.3458)
- Negative prices (e.g., one Apple stock is worth $-200)
- Dummy and test quotes (submitted to test latency or other technical specifications)[4]
- Extra or removed digits (e.g., $10000 instead of $1000)
- Invalid date (e.g., option maturity on 01-01-1345)
- Inverted exchange rates (e.g., 1 dollar equals 1.27 pounds instead of 1 pound equals 1.27 dollars)
- Rounding (e.g., price of $1.01 rounded to $1)
- Misspelled entity names (e.g., Bnk of America)
- Typos (e.g., $0900)

- Invalid formatting (e.g., 01-2022-01)

Let's walk through an example. Suppose you want to convert one billion euros to dollars. Let's assume that the Forex quote you are supposed to use is 1 EUR = 1.07291 USD. Using this exchange rate, the converted sum is $1,072,910,000. Now, assume that the exchange rate is slightly different due to a data error, say 1.07191. In this case, the newly converted sum is $1,071,909,999, which is $1,000,000 less! Similarly, if a decimal precision error happens, say the exchange rate is 1.072, the converted sum would be $1,072,000,000, leading to a loss of $910,000.

NOTE

An important aspect to keep in mind about financial data is the nature of its correctness or trueness. While certain financial variables possess an absolute and indisputable value in specific scenarios—like the number of shares sold in a market transaction—others, such as derivative prices or Forex quotes, are subject to estimates, averages, or provider-specific values. For instance, a EUR/USD quote may differ among Forex brokers, with no universal market quote for reference. Therefore, it's vital to assess financial data errors against the appropriate reference value.

Data errors can also significantly impact the robustness of financial analysis. For instance, a research article from the *Journal of Fixed Income* estimates that around 7.7% of transaction reports in the Trade Reporting and Compliance Engine (TRACE) database are erroneous records. If these errors are not considered, liquidity measurements based on this data may be skewed toward indicating a more liquid market than is the case.

To handle financial data errors, a few steps are required. In the first step, you need to identify and detect data errors. Error detection in financial data can occur at either the single-record or dataset level, with the former focusing on individual data points (e.g., an error with a single transaction) and the latter analyzing multiple records simultaneously, often producing aggregated error metrics like the error ratio (e.g., for statistical analysis purposes).

Crucially, financial data errors vary in complexity, making detection difficult at times. For example, a computer algorithm can easily detect an intraday price jump from $100 to $0.100. However, a more subtle error might require a more in-depth investigation, such as an intraday price of $50, followed by three prices of $40 and then a fifth price of $50. For simple errors, rule-based approaches are often used (e.g., if the price is negative → error). For more complex errors, statistical and data mining techniques have been traditionally employed, such as Pearson correlation, z-score, percentile analysis, and Mahalanobis distance.[5] A more advanced technique involves the computation of the value of the erroneous record using theoretical or quantitative models such as financial asset pricing.

Once detected, errors need to be checked against business-defined tolerance and impact levels. A tolerance level can be something like an error ratio < 0.01%. Once the error is checked against the tolerance ratio, the next action depends on its business priority. An error with a Forex exchange rate may significantly impact the business if it converts large sums of money and, therefore, it needs to be given high priority.

A challenging situation arises when the data containing errors comes from a third party and is not produced by the final data consumer. In this case, detecting and correcting the errors might be difficult as there is no valid ground truth to compare the data against. When this happens, a useful approach is cross-dataset validation, which consists of comparing data from one source against an alternative data source that records similar but high-quality data. In a research article from the *Journal of Finance*, the authors analyzed error rates in CRSP (a stock price dataset) and Compustat (a company fundamentals dataset) and found that errors happen with a very low frequency, but the impact of existing errors is substantial. In the same paper, the authors suggest that their methodology could be generalized as a means of data quality assessment for competing databases.

Dimension 2: Data Outliers

In basic terms, a data outlier is a data observation that differs significantly from the others. For example, consider a stock price time series with 100 observations, 99 of which have a value less than 1000, but one record has a value of 1,000,000. It is typical to refer to this last observation as an outlier. The presence of outliers in financial data may adversely affect the robustness of statistical analysis and bias machine learning models.

Outliers within financial data might arise due to various factors. In market and transaction time series, outliers commonly result from the inherent high noise level in the data. Additionally, outliers may signal fraudulent or anomalous financial activities like money laundering and credit card fraud. Furthermore, some records may seem like outliers due to systematic issues (e.g., data transmission errors), structural breaks (sudden shifts in market conditions), poorly formatted or unadjusted data, or measurement errors (e.g., errors in price quoting).

To identify financial data outliers, researchers have proposed several methods. Some use statistical techniques such as principal component analysis, z-score, percentile analysis, and kurtosis, while others use machine learning techniques such as clustering, classification, and anomaly detection.[6] Keep in mind that financial outlier detection might be a challenging task whose difficulty depends on the type and structure of the data. For example, outliers in financial time series data are different in terms of detection and treatment than in cross-section data.[7]

Once detected, outliers need to be treated following a specific method. The most common outlier treatment methods among financial researchers are *winsorization* and *trimming*. Trimming is the simplest approach, which works by removing outliers from the dataset. The main challenge with trimming is that if you trim too much, you risk altering the statistical properties or coverage of the dataset, while if you trim too little, you might still end up with an unstable and noisy dataset.[8]

Winsorization, on the other hand, involves limiting extreme values in the data to a specified percentile. In a 90% winsorization, for instance, all observations above the 95th percentile are set to equal the value of the 95th percentile, and all observations below the 5th percentile are set to equal the value of the 5th percentile.[9]

Another popular and reliable technique is *scaling*. This can be performed by taking a dataset's logarithm or square root value. Scaling helps to normalize the data distribution and reduce the impact of extreme values. For instance, consider a dataset containing stock prices where some stocks have significantly higher prices than others. By applying logarithmic scaling to the stock prices, the dataset's range can be compressed, making it easier to compare and analyze percentage changes or returns across different stocks. This normalization step helps in recognizing trends and patterns without being heavily impacted by the extreme values of high-priced stocks.

Another technique involves trimmed estimators, which are statistical measures created by excluding a portion of the extreme values from the dataset through truncation. For instance, the 5% trimmed mean is computed by averaging the values within the 5% to 95% range, removing the lowest and highest 5% of the data.

Dimension 3: Data Biases

Data biases refer to inherent distortions that impact the representation of the subjects/entities that constitute the data. These biases can result in erroneous conclusions, skewed patterns, biased decisions, and discrimination.

A large number of biases may emerge in financial data. To illustrate with an example, let's say you want to analyze the performance of fund managers (a common area of study in finance) and get a dataset for your study from a data provider. Interestingly, many commercial fund datasets are collected via a voluntary reporting mechanism. In this setting, fund managers may or may not decide to report their performance highlights to the data vendor.

For instance, when a fund performs well, and the manager aims to attract more capital, they may disclose its figures to draw market attention. Conversely, if the fund underperforms or the manager prefers not to attract new investors, they might opt not to report the performance figures. This kind of behavior might lead to self-selection bias. Such bias will distort the dataset and give the impression that some funds are always outperforming.

As institutional investors, hedge funds are exposed to a number of risks, and in some cases, this might lead to failure/bankruptcy. If a hedge fund dataset systematically excludes/removes failed or poorly performing funds from its archive, this could lead to a type of bias called *survivorship bias*, where only the successful funds appear in the dataset. Similar to self-selection bias, survivorship bias may convey an over-optimistic image of the fund industry's performance.

In some cases, a new hedge fund could be added to the dataset together with its full history. Even though it looks natural, this behavior might lead to *backfilling* bias, also known as *instant history bias*. This bias can be relevant if only funds with a strong track record choose to join the database, which would distort historical performance statistics for the hedge fund industry.

Another significant bias in financial data is *look-ahead bias*, which occurs when conducting historical studies using information that would not have been accessible during the analyzed period. For instance, consider a scenario where a company releases its annual report for the year 2019 in March 2020, as is typically the case. Suppose you're a financial analyst conducting backtesting to evaluate your investment strategy's performance. In that case, it's crucial to avoid assuming that the company's annual report was available before March 2020 (e.g., December 31, 2019), even if you're analyzing data from a later date.

Detecting and correcting biases in financial data is a challenging task. As a financial data engineer or analyst, make sure you understand how the data was generated and recorded. Consider collecting more data to adjust for possible biases. Furthermore, if you are extracting a data sample from a large dataset, design a methodology that proactively prevents biases, for example, by assessing the data sample representation. Another good practice is to compare two datasets to check for potential bias that exists in one but not the other.[10] Finally, I highly recommend keeping an eye on the latest research on bias and fairness in data analysis and machine learning.[11]

On the data vendors' side, efforts have been made to detect and correct biases in their products. For example, among the most reputable hedge fund data sources is LSEG's Lipper Fund Data, formerly known as the *Trading Advisor Selection System* (TASS) database. Lipper tracks and publishes fund-related information such as its profile, performance, and investment strategies. Funds report to Lipper voluntarily, which, as we saw earlier in this section, can lead to various biases. To account for this issue, Lipper keeps two separate databases: the *graveyard* database, which records data on defunct funds or funds that haven't been reported for a long time, and the *live* database, which records data for actively reporting funds.[12]

Dimension 4: Data Granularity

Data granularity describes the level of detail within a dataset, with highly granular data offering detailed observations about individual entities, while low-granularity data typically provides summarized or aggregated information at a higher level. To better illustrate the concept within the finance domain, let's consider a few examples:

Financial portfolios

A financial portfolio is a collection of investments created to achieve a specific financial goal, considering elements such as diversification, risk appetite, and expected returns. Portfolio data can be available either in aggregated form, providing an overview of portfolio performance, risk, and investment strategy, or in a more detailed form, including details about individual portfolio constituents and their respective allocations, such as Apple stock at 5%, US government bonds at 20%, and so on.

Financial indices

A financial index is a single aggregated metric constructed to represent and track the performance of a specific category of financial assets. For example, the S&P 500 index offers an aggregated metric

of the top 500 public companies in the United States by market capitalization. Index data may be available at the index level (single metric) or at the constituent level (the individual assets included in the index and their corresponding weights).

Financial exposures

Financial institutions hold assets with each other, such as interbank loans, securities, and cash. A financial institution might disclose its total exposure to another institution at the aggregated level (Bank A holds $1bln assets with the rest of the system) or at the individual institution level (e.g., Bank A holds $1mln at Bank B, $33mln at Bank C, and so on).

Financial time series

These can be recorded with high temporal accuracy, such as every second, minute, or hour, or with lower granularity, such as daily, weekly, or monthly aggregates.

Financial transactions

These can be stored at the individual transaction level or as summaries, such as monthly purchases.

The level of data granularity is an essential factor in determining the type of analysis that can be performed on the data. Highly granular data enables deeper insights and the identification of meaningful patterns utilizing advanced analytical approaches. Importantly, granular data comes with increased storage requirements, processing time, and the potential for privacy concerns. Therefore, it is recommended to keep in mind the challenges and tradeoffs associated with managing and analyzing highly granular data.

Granular financial data may not always be available. Portfolio composition data, for example, may not be disclosed since it would reveal the firm's investment strategy, which might harm its competitive position. Another reason this information may be withheld is data confidentiality, like with customer transaction details. In some cases, detailed data may not be collected in the first place, for example, in noncentralized markets such as OTC markets.

To overcome the issue of data granularity, you can try to collect detailed data. Alternatively, aggregated data may be decomposed into its constituent elements using statistical and machine learning techniques. For example, network scientists who analyze financial networks are often limited by privacy issues when collecting data about the topology of a financial network. As mentioned earlier, the best example is banks' exposures to each other, where data is available at the aggregate level only. To overcome this issue, researchers have proposed network reconstruction and link prediction methods to infer and construct the network structure at the bank-to-bank level.

Dimension 5: Data Duplicates

Data duplicates are repeated records that represent the same data observation. Duplicate data is a widespread issue in finance, and its consequences can range from negligible to severe. For example, a duplicate record in a data sample used in a financial study may not lead to serious

consequences; however, if a financial transaction appears multiple times on a user account, then it might impact the available balance.

Data duplicates may occur for a variety of reasons. First, there are human errors, which happen when a person adds the same data entry into a system multiple times. Second, duplicates may be inserted automatically by a machine when the system is not properly built to ensure data uniqueness. For example, a household submits a loan application twice. Third, data duplicates might emerge when merging multiple data sources improperly, for example, using nonunique identifiers.

Crucially, despite the apparent simplicity of the problem, detecting duplicates may be quite challenging. In the simplest case, a data record is considered duplicate if the values of all its fields exactly match those of another record. For example, two financial accounts are considered duplicates if they match the account holder's first name, last name, social security number, and account number. In a more complex scenario, two duplicate records may be recorded differently, thus making it harder to identify. For example, if names are recorded with different formatting (e.g., J.P. Morgan versus JPMorgan), then the duplicate records would have a nonperfect match. In other cases, the presence of exact duplicates may be due to inconsistency in the data recording mechanism. For example, an investment of $10,000 in stock A may be recorded as an investment of $5,000 in stock A twice, while an investment of $10,000 in stock B is recorded only once.

Detecting and treating duplicate records can also vary in complexity. The best strategy is to always think in advance about the duplicate generation mechanism and add the necessary checks, constraints, and validations to prevent duplication. For example, having a well-defined unique identifier for each record is a good practice. If the unique identifier is not sufficient, then it is possible to add constraints on a subset of data fields that ensure no two records share the same set of values. Let's walk through an example of how to prevent duplicates before data insertion in PostgreSQL:[13]

```
-- PostgreSQL
CREATE EXTENSION btree_gist;
CREATE TABLE company(
   record_key INT PRIMARY KEY,
   company_id VARCHAR NOT NULL,
   company_name VARCHAR NOT NULL,
   dividend_date DATE NOT NULL,
   dividend_amount DECIMAL(10,2) NOT NULL,
   EXCLUDE USING gist (company_id WITH =, company_name WITH <>)
);
INSERT INTO company VALUES(1, 'JPM', 'JP Morgan', '2020-11-20', 10);
INSERT INTO company VALUES(2, 'BOA', 'Bank of America', '2022-01-08', 5);
INSERT INTO company VALUES(1, 'JPM', 'JP Morgan', '2019-05-01', 8);
INSERT INTO company VALUES(3, 'JPM', 'J.P. Morgan', '2023-06-01', 3);
```

```
psql:commands.sql:12: ERROR:  duplicate key value violates unique constraint "company_pkey" DETAIL:  Key
psql:commands.sql:12: ERROR:  conflicting key value violates exclusion constraint "company_company_id_co
DETAIL:  Key (company_id, company_name)=(JPM, J.P. Morgan) conflicts with existing key (company_id, comp
```

In this example, we created a table that stores data on company dividend distributions. Each company has a unique ID and a human-readable name, while each record has a unique record key. We want to avoid hav-

ing two records with the same record key or the same company ID but with a different company name. To achieve this, we can add a primary key constraint on the record key field and an *exclusion constraint* on the company ID and company name. After that, we test the implementation by trying to make four inserts, two of which violate the two constraints.

The first two insert statements will execute successfully, and two records will be created. However, for the third statement, we will get an error that says the uniqueness constraint was violated as they share the same record key (= 1). For the fourth insert, we will get an error saying that you are trying to insert a record with a new format (J.P. Morgan and JP Morgan).

If data has already been generated and stored with potential duplicates, then a variety of solutions are possible. In the simplest case, duplicates share the same field values. To identify them, we can use aggregation or analytical queries. Let's consider the following example:

```
-- PostgreSQL
-- create
CREATE TABLE company (
  company_id INTEGER PRIMARY KEY,
  company_name VARCHAR,
  company_headquarters VARCHAR
);

-- insert
INSERT INTO company VALUES (1, 'Company A', 'New York');
INSERT INTO company VALUES (2, 'Company B', 'California');
INSERT INTO company VALUES (3, 'Company A', 'New York');

-- fetch
SELECT company_name, company_headquarters, count(company_id) AS record_count
FROM company
GROUP BY company_name, company_headquarters
```

```
 company_name | company_headquarters | record_count
--------------+----------------------+--------------
 Company B    | California           |            1
 Company A    | New York             |            2
```

In the above table, two duplicates store the same data for Company B. To detect these duplicates, we used a `GROUP BY` statement that counts the number of records that share the same company name and company headquarters.

The `GROUP BY` is an aggregation tool that reduces the number of rows to summary groups. If we want to keep the original data without aggregation, we can use a window function such as `ROW_NUMBER()` to assign an ordered sequential number to each record in a group. This way, deduplication can be performed by taking the records with `row_num` = 1 and discarding those with higher row numbers. Here is an illustrative example:

```
SELECT company.*,
ROW_NUMBER() OVER (PARTITION BY company_name, company_headquarters) AS row_num
FROM company
ORDER BY company_name, company_headquarters, row_num
```

```
 company_id | company_name | company_headquarters | row_num
------------+--------------+----------------------+---------
          1 | Company A    | New York             |       1
          3 | Company A    | New York             |       2
          2 | Company B    | California           |       1
```

A more challenging scenario occurs when a dataset contains duplicates, but they cannot be directly identified due to data quality issues. For example, we might know the subset of columns that could identify duplicates, but the data may be recorded differently, so it won't be detected via an exact match. In this case, the data deduplication process becomes an entity resolution (data matching) task. We discussed entity resolution systems in Chapter 4. One thing to keep in mind is that data deduplication is a special type of entity resolution, as it involves only one dataset that is matched against itself to resolve similar entities.

Another subtle scenario with duplicates happens when two records look the same, but they are not duplicates because a distinguishing field is missing. Examples include transaction data where the date of the transaction is available but the time is missing; multiple data at the security level (e.g., company stocks) that look like duplicates but, in reality, refer to different security issues (e.g., different stock issues) that are missing issue IDs; or a financial option is available twice, but one is a call (buy-side), and another is a put (sell-side).

Dimension 6: Data Availability and Completeness

A crucial dimension of data quality is the completeness of a dataset, indicating whether it contains all necessary information required for its intended analytical or operational purposes. A dataset is considered incomplete or unavailable when essential data attributes or observations are missing. In finance, issues of data availability and incompleteness are quite common. This happens for a variety of reasons, such as the following:

Voluntary data reporting

If the data collection process involves a voluntary data reporting mechanism, e.g., survey data, then it is likely that some respondents will decline to report their data for one reason or another, e.g., to hide bad performance. Additionally, respondents might report data with different frequencies (e.g., Firm A responds to all surveys, while Firm B responds to some and skips others).

Security and confidentiality concerns

Unless enforced by law, financial firms might have a number of concerns about the security and confidentiality of their data. This might generate a high level of risk aversion toward data sharing.

Market factors

Market factors such as liquidity, sentiment, and risk might impact the data generation process. For example, a liquid stock that trades frequently will have a large number of price observations per day, while an illiquid instrument might trade once every six hours and have a few daily records. This type of behavior is called nonsynchronous trading.

Technological reasons

If a financial firm or the market lacks adequate data collection infrastructure, it can lead to data collection gaps. For instance, a considerable amount of data on over-the-counter (OTC) market transactions remains unrecorded due to the absence of a centralized entity responsible for collecting and aggregating such data.

Publication delay

If there is a latency between the time data is created and the time it is published, it could be considered unavailable. This might happen with company fundamental data, created at the end of the fiscal year but released a few months later.

Data Time to Live (TTL)

TTL is a database mechanism that sets a period of time after which data will be considered expired and no longer visible to queries and database statistics. TTL doesn't necessarily mean that the data was deleted, as this might happen at a later point in time.

In certain circumstances, the existence of a specific type of data may be optional; in these situations, the consequences of missing data are minor and may not require any correction effort. However, in many cases, incomplete or missing data can cause a number of problems for financial institutions. For example, missing data may lead to biased and unreliable financial models, which in turn can impact investment decisions and product development. Incomplete data may also mean less visibility and insight into market activities and patterns, thus foregoing potentially profitable opportunities and reducing trust in the data within a financial institution. Gaps in customer-related data may impact the business and sales teams' ability to understand consumer segments and offer personalized services. Moreover, missing data may delay reporting and releases, which can impact market sentiment and expert estimations.

To effectively deal with missing financial data, it is crucial to understand the mechanisms that cause data missingness. Following the existing literature, three forms of missing data are often discussed:

Missing Completely at Random

Data on variable X is said to be Missing Completely at Random (MCAR) if the mechanism that leads to observations of X being missing is independent of X itself and any other variable in the dataset, whether observable or missing. The missingness happens randomly and without any systematic pattern. For example, a hedge fund that reports its performance data to a data provider might incur a technical issue preventing some of its data from being submitted.

Missing at Random

If data on variable X is Missing at Random (MAR), then the missingness mechanism is independent of X itself but is systematically related to one or more features in the dataset. For instance, a hedge fund firm might opt not to disclose performance data due to confidentiality concerns surrounding its investment strategy. Here, the missingness is unrelated to the fund's performance but rather to its investment secrets.

Missing Not at Random

> This happens when observations on variable *X* are missing for reasons related to *X* itself. To continue with our example, a hedge fund might decide not to report its performance data because it had a bad performance and wants to hide it from investors or because they are doing very well and they don't want to attract more investors or media coverage.

A variety of techniques have been developed for treating missing financial data. The simplest and most common technique is *likewise deletion*, where observations with missing data are removed from the dataset. Another technique is the *omitted variable approach*, which drops the variable with missing values from the dataset. In some cases, dropping observations or variables might lead to biased or sparse datasets. A family of techniques called *imputation* is often used to overcome this issue. Imputation aims to estimate the missing values in a dataset using a specific method. One basic imputation technique is the *mean substitution*, which replaces missing values for variable *X* with the average value of *X*. Another imputation technique is filling in a missing value in one dataset with another value in another dataset, e.g., via entity resolution. Last but not least, a practical approach to data imputation is regression, where a model is used to produce an estimate of the missing value. Models can be machine learning based (e.g., linear regression) or financial models (e.g., Capital Asset Pricing Model, Value at Risk, etc.).

Dimension 7: Data Timeliness

Timeliness of data is a critical dimension of data quality within financial institutions. This section will discuss two key aspects related to data timeliness:

- Is the data available and accessible at the time it is expected to be?
- Does the data reflect the most recent observations?

Many financial datasets are used in a time-critical context, e.g., algorithmic trading, and if data is not available in the expected window, then the data is of no use. On the other hand, if the available data does not reflect the latest facts, e.g., the latest Forex quote or the latest analyst estimate, then it might lead to wrong business decisions and lost revenues. Financial markets use the term *stale price* to describe an outdated or no longer accurate quoted value of a financial asset or instrument.

A variety of factors may influence financial data timeliness. The most common factors are latency, market closure, time lags, or lengthy processes in the data generation, ingestion, and transformation mechanisms. For example, complex data pipelines are likely to be time-consuming and delay data availability. Another factor is the data *refresh rate*, which is the frequency with which data is refetched and updated to reflect the latest observations. Refresh frequencies may vary from real time to regular schedules.

Furthermore, many applications rely on *data caching*, a strategy where a copy of the data is stored in a temporary storage location that allows fast access. However, cached data can become outdated over time, requiring periodic updates or replacements to ensure alignment with the most re-

cent data. This problem, known as *cache invalidation*, poses one of the most common challenges in software development.

Dimension 8: Data Constraints

The data constraints dimension reflects the degree to which data conforms to predefined technical and business rules and limitations. Examples of such constraints include the following:

Extension constraint

Data is stored in allowed formats only (e.g., CSV files).

Schema constraint

Data follows a predefined schema structure that defines the mandatory fields and their data type.

Non-null constraint

A data field does not contain null values.

Range constraint

A data field contains values that fall within a given range (e.g., price >=0, year >=1990).

Value choice constraint

A field can assume values from a fixed list of choices (e.g., country names).

Uniqueness constraint

A record must be unique across a dataset.

Referential integrity constraint

Values in one field are allowed only if they exist in another referenced field. For example, an online purchase transaction cannot be stored if it contains a nonexistent product.

Regular expression patterns

A field contains values that match a given string pattern (e.g., an email pattern, a financial identifier pattern).

Cross-field validation

This ensures that a field satisfies a certain condition in relation to one or more fields. For example, the date of issuance of a derivative contract cannot be earlier than the date of expiry.

Based on your business needs, additional constraints may be defined. An important thing to remember is that violating a given constraint may not always signal a bad data quality issue. For example, a schema change that involves adding or deleting a given field might be done to enrich data quality and correct existing errors. As another example, a value choice constraint may be violated if the list of allowed options is outdated.

Dimension 9: Data Relevance

Data relevance is an important data quality dimension, determining the degree to which available data aligns with the specific problem or purpose it aims to address. Relevance ensures that the data is actionable and contributes effectively to gaining insights and understanding the problem at hand. For example, in his interesting analysis published in *Getting It Wrong: How Faulty Monetary Statistics Undermine the Fed, the Financial System, and the Economy* (MIT Press, 2011), William Barnett illustrates how the lack of adequate financial data to assess financial systemic risks has been identified as one of the main factors leading to the great financial crisis of 2007–2008.

In the following years, several initiatives have been launched to review and enhance the data collection processes in financial markets. For example, in 2020 the Bank of England conducted a Data Collection review to identify the challenges the industry faces in providing data, the issues the bank encounters in receiving and using it, and the necessary steps to address these problems. As another example, in response to the significant impact of swaps, especially credit default swaps, during the 2007–2008 financial crisis, the Dodd-Frank Wall Street Reform and Consumer Protection Act (Dodd-Frank Act) established *swap data repositories* (SDRs) to serve as entities responsible for swap data reporting and recordkeeping. According to the Dodd-Frank Act, all swaps, whether cleared or uncleared, must be reported to registered SDRs.

In recent years, the importance of data relevance has increased alongside the rise of machine learning and generative AI. If the available data features (variables) are not pertinent to the analytical problem or do not provide insights into the patterns analysts aim to capture, developing accurate models becomes challenging.[14] Similarly, fine-tuning a language model requires contextual data that matches the specific requirements and conditions of the task or problem at hand.

The question of what data is relevant is ultimately a function of the problem at hand. For example, investment firms may employ a range of trading strategies, each of which requires different types of data. For instance, *day trading* demands real-time intraday price, volume, volatility, and market liquidity data. *Swing trading* relies on technical and fundamental indicators, market sentiment, and medium-term price trends. *Trend trading* depends on moving averages, trend lines, and momentum indicators. *Arbitrage trading* needs data on order books, market liquidity, transaction costs, trading fees, real-time price discrepancies, and Forex rates. *Mean reversion trading* uses data on moving averages, Relative Strength Index (RSI), Bollinger Bands, and Moving Average Convergence Divergence (MACD); *systematic trading* requires time series data for transactions (price and volume), orders, and news (economic releases and events).[15]

Data Integrity

Throughout its lifecycle, data goes through a number of transformations, movements, and adjustments as well as aggregation, matching, and more. In this context, the concept of *data integrity* is often used to indicate a set of principles established to ensure consistent, traceable, usable, and reliable data. Depending on the institution type and business requirements,

there may be a number of ways data integrity could be ensured. To offer a general overview, this section outlines nine key data integrity principles —standards, backups, archiving, aggregation, lineage, catalogs, ownership, contracts, and reconciliation—all of which hold significant relevance within the financial sector.

Principle 1: Data Standards

As financial markets have grown in size and complexity, the terms *standard* and *standardization* have become keywords. According to authors Spivak and Brenner in their book *Standardization Essentials* (CRC Press, 2001), the term standard "denotes a uniform set of measures, agreements, conditions, or specifications between parties." The process of formulating, developing, and implementing standards is called standardization. Standards differ in their nature, function, and acceptance. Spivak and Brenner provide a general framework by categorizing standards into the following taxonomy:

- Physical standards or units of measure
- Terms, definitions, classes, grades, ratings, or symbols
- Test methods, recommended practices, guides, and other applications to products and processes
- Standards for systems and services, in particular, quality standardization and related aspects of management system standards for quality and the environment
- Standards for health, safety, consumers, and the environment

Financial industry participants have made several calls for standardization, acknowledging its role in increasing market efficiency, confidence, and stability and reducing costs. For example, research conducted by McKinsey found that the adoption of the Legal Entity Identifier standard (which we discussed in detail in Chapter 3) could save the global banking industry around USD $2–4 billion annually in client onboarding costs.

Furthermore, standards play a crucial role in financial markets by promoting consistency across key aspects of processes, products, and services, including quality, compatibility, interoperability, comparability, and reliability. For instance, the ISO 21586 standard, which specifies the description of banking products or services, was introduced to ensure uniformity in descriptions of banking products and services (BPoS) across various financial institutions, enabling customers to understand and compare them effectively.

Furthermore, as domain experts develop standards, they distill best practices and codify the most recent technologies and expertise, which saves market participants the effort of reinventing the wheel. A few great examples of this are accounting standards (e.g., *generally accepted accounting principles*, or GAAP), risk management standards (e.g., value at risk, or VAR), and data quality standards (e.g., ISO 8000: Data Quality).

Principle 2: Data Backups

One of the primary operational risks faced by financial institutions is the loss of data, which can occur for various reasons. For example, data may be destroyed by accident, corrupted, converted incorrectly, overwritten with a later version, mixed with the incorrect sort of data, lost during a

hardware failure, or simply buried in a large pile of log files where it is hard to detect.

Data backups are an effective method for mitigating the risk of data loss. A data backup is a copy of the original data stored in a different place that can be recovered in a case of a data loss accident.

A number of factors need to be considered when building a data backup strategy. I recommend approaching the problem through a *data backup lifecycle*. This includes defining data backup steps and elements such as when to back up (e.g., scheduled, on demand, or event driven), what data to back up (operational, analytical, client), the number of backups to create, backup security (e.g., through encryption), backup storage locations (multizones, geographies, data centers), recovery tests and plans, retention time, and deletion.

Principle 3: Data Archiving

A variety of data within financial institutions may reach a point where they are no longer actively utilized but are still necessary for future reference, reviews, audits, electronic discovery, litigation, and compliance purposes. Examples include financial transactions, customer information, and regulatory reports.

In addition, regulatory frameworks such as the Bank Secrecy Act (BSA), Federal Deposit Insurance Corporation Improvement Act (FDICIA), and General Data Protection Regulation (GDPR) have established several data retention requirements that financial institutions need to comply with.

To manage this data retention challenge, a common approach is *data archiving*, which refers to the process of moving data that is no longer actively used out of production systems and onto a separate long-term storage system.

NOTE

Data archives should not be confused with data backups. Although both are concerned with keeping data in secondary storage, they serve different purposes. Data backups are part of a disaster recovery strategy and are meant to manage data loss accidents. Data archives, on the other hand, serve a data retention purpose.

In addition to compliance and risk management, data archiving lowers storage costs by moving data from more costly high-performance primary storage to significantly less expensive secondary storage (e.g., hard disk drives [HDDs]).

A *data archival policy* is often established to manage data archival. It comprises elements such as a data retention policy, data archival software, and data access and discovery functionalities.

Principle 4: Data Aggregation

Financial institutions engage in diverse operations, including lending, payments, investments, risk management, insurance, proprietary trading, portfolio management, and more. Often, a single institution performs

multiple activities simultaneously. For instance, a commercial bank may accept deposits, offer loans, market financial investment products, and provide insurance.

In traditional settings, individual activities within a financial institution are often overseen within distinct organizational silos, each maintaining data about its operations using separate systems. An inherent challenge with such silos is the complexity involved in consolidating data across a large number of business units, legal entities, and disparate data storage systems. This, in turn, may jeopardize a financial institution's capacity to generate an aggregated view of its activities and risks. Such constraints were one of the primary reasons that banks could not adequately assess their risk exposures and concentration before and during the 2007–2008 financial crisis.

NOTE

One thing to keep in mind is that data aggregation is a capability of a data infrastructure, and it doesn't necessarily mean having all data in the same place.

In response, the Basel Committee published a set of 13 principles for designing and implementing data aggregation capabilities in financial institutions, primarily banks. Logically, the final implementation of these principles will vary from one financial institution to another. In addition, variations in internal structures among banks may facilitate or hinder the complete implementation of all 13 principles. Indeed, in assessing the adherence to the guidelines, the Basel Committee observed that banks were not fully compliant, mainly due to the complexity of their internal IT infrastructure.

Several other practices in financial markets emphasize the aggregation of data. For instance, asset and fund managers typically maintain an *Investment Book of Record* (IBOR), a centralized database consolidating investment-related information like transactions, positions, and holdings. Another example is the *Accounting Book of Records* (ABOR), which aggregates accounting-related data on assets, liabilities, transactions, costs, net asset value, and charts of accounts. Another good example are *consolidated tapes*, which refer to systems that aggregate data from different trading venues, offering a unified source of information on trading activity across multiple markets.

Principle 5: Data Lineage

As a financial data engineer, a practical way to think about data is through the *data lifecycle* approach, also called the *information lifecycle*. The term data lifecycle is frequently used in industry to indicate the different phases that data goes through from the initial creation, or ingestion, onward. Based on each financial institution's particular context, the data lifecycle's phases and complexity may vary. At a high level, there are five main phases: extraction, transformation, storage, usage, and archiving. However, this process can quickly evolve into a complex chain of steps involving a multitude of actions and operations. Consequently, maintaining visibility into the data lifecycle becomes increasingly challenging, potentially exposing financial institutions to costly risks and errors.

To gain visibility into the data lifecycle, financial institutions need to develop a *data lineage* framework. The term data lineage is used to describe the discrete steps involved in the generation, movement, transformation, storage, delivery, and archiving of data. In other words, it is a feature of the data infrastructure that allows users and engineers to track a given data object throughout its lifecycle. Knowing how the data was generated and what actions were applied to it at each step builds confidence in the data infrastructure, pipelines, and lifecycle. Nowadays, the visibility of data lineage is a valuable feature that is top of mind for consumers, managers, and regulators.

Data lineage can be implemented in a number of ways. The most appealing and user-friendly approach is *lineage graphs*, which display a graphical visualization of data processing history. Importantly, visualization tools may not be performant when the processing logic becomes more complex and intricate. In such cases, detailed step-by-step descriptions of the processing logic are required.

AUDIT TRAILS: A FINANCIAL DATA LINEAGE APPROACH

When working in finance, a common term that you might frequently encounter is *audit trail*. An audit trail is a special implementation of data lineage that builds a chronological step-by-step data recording system that tracks financial activities such as accounting transactions, financial transactions, trades, buy and sell quote submissions, and any financial activity that can be tracked from its origination onward. An audit trail is often used when an auditor or regulator wants to examine the origin of a certain figure (e.g., earnings per share) or a given financial activity (e.g., quote to sell a specific number of securities). The importance of audit trails has increased remarkably after the flash crash of 2010, where a trader intentionally submitted an exceptional set of orders (a practice called *spoofing*) in order to manipulate the market in their favor. Thanks to audit trails, regulators were able to identify the person responsible for the orders. Systems such as the Order Audit Trail System (OATS) and Consolidated Audit Trail (CAT) were established to automate the recording of information on orders, quotes, and other trade-related data from all shares traded on the National Market System (NMS). Such systems streamline the lifecycle of an order from reception through execution or cancellation for simple tracking and auditing.

Principle 6: Data Catalogs

Financial institutions produce and consume vast quantities and varieties of financial, economic, operational, and business data. To create an efficient and reliable data-driven culture, data producers and consumers must be able to search and find all the data they need quickly.

In this context, the idea of *data catalogs* has received particular attention. In simple terms, a data catalog is a set of metadata (i.e., data that describes or summarizes other data) combined with search and data management tools that allow a data producer or consumer to find and document a data asset within a financial institution. In other words, you can think of a data catalog as a central and searchable inventory of all data assets.

Data catalogs can be implemented in a variety of ways based on your data consumption needs and the complexity of your data assets. For instance, it can be a database where you store and search the metadata directly or a full-fledged application with features such as a UI, search and discovery, metadata management, user permission, and API integration. For a practical example of such tools, have a look at the open-source Python-based library Comprehensive Knowledge Archive Network (CKAN). To see a minimal data catalog in action, check the online data catalog of LSEG Data & Analytics.

Crucially, the more advanced and complex a data catalog, the higher the maintenance and curation burden. Conversely, having a sparse or out-of-date data catalog may lead to higher resource use and more wasted time than not having any.[16]

Principle 7: Data Ownership

Data ownership is one of the most valuable data governance practices for data-driven organizations. It's important to note that the term data ownership is used in two different contexts. On the one hand, it can refer to the legal owner of a given data asset. This topic emerged following the considerable increase in data collection practices and the adoption of third-party storage solutions such as the cloud. As more and more data is collected about people and organizations, concerns have emerged regarding who the final owner of the data is. Similarly, with the widespread adoption of public cloud storage solutions, questions have emerged regarding who owns the data stored in the cloud.[17]

Looking at it differently, data ownership involves designating an individual or team with the task of overseeing the collection, cleansing, maintenance, sharing, and management of a particular data asset within a financial institution. These individuals are often known as *data owners* and are typically selected for their domain expertise. The rationale is that data owners, being subject matter experts, are better equipped and motivated to maintain and manage a specific data asset compared to a centralized team that may lack the requisite domain knowledge to comprehend the data fully.[18]

Principle 8: Data Contracts

One of the main factors that can significantly impact the quality of data within organizations is the communication structure. A well-known adage, called *Conway's law*, is often cited in this context. It states that "organizations which design systems are constrained to produce designs which are copies of the communication structures of these organizations." Following this principle, the term *data contract* has recently emerged as a promising approach to organizing requirements and expectations around data. Let's consider how industry experts define data contracts:

> *Data contracts are API-like agreements between Software Engineers who own services and Data Consumers that understand how the business works in order to generate well-modeled, high-quality, trusted, real-time data.*
>
> — Chad Sanderson, "The Rise of Data Contracts"

> *A data contract is an agreed interface between the generators of data and its consumers. It sets the expectations around that data, defines how it should be governed and facilitates the explicit generation of quality data that meets the business requirements.*
>
> — Andrew Jones, *Driving Data Quality with Data Contracts* (Packt, 2023)

With a data contract, data consumers can define their data-related needs (e.g., structure, semantics, relations, formatting, fields, frequency, typing, rounding, privacy, and terms of use) and establish an agreement with data engineers to receive data that matches their expectations. This allows data consumers to concentrate on analysis and product development rather than worrying about data generation and engineering. Data engineers, on the other hand, do not need to be concerned about making modifications to the database or data model that may lead to production issues. This is why it is called a contract: both sides agree to it and each needs to hold their end of the deal. Naturally, a data contract can be revised and changed (e.g., a new field is required, a new owner is assigned, etc.) which would result in a new modified agreement.

Currently, general guidelines for implementing data contracts don't exist. Depending on the institution and its data strategy, different data contract definitions may be established.[19] To give an illustrative example, assume that the analytics team needs daily price data for the top 100 US stocks by market capitalization, with no null prices, a price range of 0–1000000 and no missing observations, and the data must be ready by 10:00 a.m. on each working day. In this case, following the data contract specification proposed by Jochen Christ and Simon Harre, we can build a basic contract as follows:

```
dataContractSpecification: 0.0.1
id: stock-price-extraction
info:
  title: Daily Adjusted Stock Price Extraction
  version: 0.0.1
  description: daily extraction of the adjusted stock price of the top 100 U.S
               stocks by market capitalization.
  owner: Analytics Team
  contact:
    name: John Smith (Analytics Team Lead)
    email: john.smith@example.com
servers:
  production:
    type: Snowflake
    project: daily_adjusted_prices_prod
    dataset: snowflake_adjusted_prices_top_100_latest_v1
terms:
  usage: Data can be used for financial analysis, backtesting, and machine
         learning use cases.
  sla: 10:00 AM of each working day.
  daily_record_count: 100 observations
```

```
    limitations: >
      Not suitable for intra-day financial time series analysis.
      Data may be missing some identifiers such as ISIN.
      Max data processing per day: 10 Gigabytes.
      Max instrument requests per day: 1000 instruments.
    cost: 0.01$ per instrument request
  schema:
    type: json-schema
    specification:
      adjusted_prices_top_100:
        description: One record per instrument and date.
        type: object
        fields:
          price_date:
            type: timestamp
            format: date-time
            nullable: false
            description: time of the price observation
          adjusted_price:
            type: numeric
            precision: 4 decimals
            range: 0-1000000
            nullable: false
            description: The adjusted price value.
          instrument_ticker:
            type: string
            description: The ticker identifier of the stock
            nullable: false
```

It's important to note that data contracts are not about specs or tools, but rather are a design pattern that emphasizes automating data quality and governance across various systems. The final specifications and the tools you will employ depend on your specific business requirements.

Principle 9: Data Reconciliation

In financial markets, the same financial record often appears across different systems owned by various counterparties, posing challenges for maintaining consistent records. This issue is addressed through data reconciliation, which involves aligning diverse sets of records to create a unified view of financial transactions and balances. This process helps minimize errors and discrepancies, thereby ensuring operational integrity, financial stability, compliance, and customer trust.

A typical data reconciliation process in financial markets is portfolio reconciliation, where records of holdings, transactions, and positions are compared and verified between two or more counterparties, such as financial institutions or investment managers. The International Swaps and Derivatives Association (ISDA) has identified portfolio reconciliation as one of the most advantageous practices in mitigating operational and credit risks in OTC markets.[20]

For instance, in the fund industry, multiple entities may hold the same portfolio exposure data for a specific fund, such as a management company and a custodian. For instance, suppose a mutual fund is managed by Management Company A, and its assets are held in custody by Custodian B. Management Company A reports a $50 million exposure in technology stocks for the fund, while Custodian B, responsible for safekeeping and reporting the fund's assets, shows a $49.5 million exposure in the same

sector. To ensure consistency and accuracy, portfolio reconciliation is necessary. This process involves comparing the data from both Management Company A and Custodian B to resolve any discrepancies and provide a unified view of the fund's holdings.

Similarly, in payment processes, multiple entities are involved in storing ledger records, often resulting in discrepancies in ledger balances among banks, FinTech companies, and service providers like BaaS providers. These discrepancies can stem from data duplication, incomplete or inaccurate record-keeping practices, delays or errors during system upgrades, and the inherent complexity of reconciling multiple systems of record.[21]

Data Security and Privacy

Financial institutions deal with highly sensitive and valuable data such as customer financial data, money transfers, transactions, investment strategies, and credit card numbers. Consequently, the financial industry has traditionally been a primary target for cyberattacks. To give a few examples, according to a report by Capgemini, Citigroup US suffered a data breach in 2011 that resulted in data on more than 360K customers being leaked. Citigroup Japan reported a similar breach that affected around 92K customers. Again in 2011, Bank of America suffered a data breach that cost the bank around $10mln.

Additionally, certain aspects of the financial industry make it particularly vulnerable to security threats. First, safeguarding intellectual property through patents and copyrights has been shown to be more challenging and ambiguous in the financial industry than in other industries such as manufacturing. Second, due to the complex interdependencies of financial systems, a security breach in one financial institution may cause a cascading shock that affects the entire system. Third, the monetary impact of a security breach at a financial institution can be consequential, given that financial data is directly connected to client funds.

Given these considerations, data security and privacy have traditionally been regarded as a top priority by financial institutions and regulatory bodies. If you have worked at a financial institution, you must have noticed that security is given significant weight in practically all discussions. Importantly, while the terms privacy and security may be used interchangeably, they actually refer to distinct problems. To illustrate the difference, I will rely on international standards developed specifically for information security and privacy.

In terms of data security, standard ISO 27001—*Information Security Management Systems* (ISMS) is the primary worldwide reference. It specifies a set of guidelines for developing an ISMS system that protects data from cyber threats. The standard guides organizations through the stages required for ISMS development, such as assessing vulnerability risks, developing policies and procedures for data protection, training employees, and managing incidents.

On the other hand, standard ISO 27701 on developing a *Privacy Information Management System* (PIMS) builds on top of ISO 27001 to ensure that a system is in place to ensure that *personally identifiable information* (PII) is handled in accordance with data legislation and regula-

tions (we discuss PII in detail in the "Data Privacy" section). The standard guides organizations through the steps needed to ensure the protection of PII, compliance with data regulations, and transparency about how organizations handle personal data.

From the two standards presented above, we can deduce that in designing for security and privacy, we consider different risks. When designing for security, we presume that an adversary may launch a cyberattack against our organization. When designing for privacy, we assume that personal data is not being handled in accordance with the law.

WHAT TYPES OF CYBERATTACKS ARE COMMITTED AGAINST FINANCIAL INSTITUTIONS?

Financial institutions can be exposed to a variety of cyberattacks. These include but are not limited to the following:

Malware

Malicious software installed on a device connected to the internal institution system. It can be a virus, spyware, Trojan, or other. If installed successfully, malware can enable the hacker to access sensitive data and compromise critical systems.

Ransomware

A special type of malware where the hacker gains access to the institution's data and holds it hostage in exchange for the payment of a ransom.

Spoofing

When a cybercriminal manages to impersonate and replicate the website of a financial institution (often banks) that looks and functions the same way. If users are not careful, they may end up providing their personal information to the fake website.

Spam and phishing

An email-based cybercrime where a person sends emails to random people soliciting them to send banking or credit card details.

Distributed denial-of-service attack (DDoS)

This occurs when a cybercriminal floods a financial institution's server with internet requests.

Corporate account takeover

When a cybercriminal gains access to a corporate account associated with a financial institution. This may allow the attacker to initiate fraudulent money transfers and other transactions.

Brute force attacks

When a hacker tries to guess the access credentials of a user via trial and error. Although it might seem outdated, it might still work if passwords are not strong enough.

SQL injection

A code injection technique where malicious SQL statements are inserted into a specific part of the application accessible to the user in order to manipulate the final query submitted to the backend database. A successful SQL injection attack might lead to data loss, a breach, or corruption.

An important thing to keep in mind is that cybercriminals are creative (newer ways of conducting cyberattacks are continuously being invented) and adaptive (workarounds are developed to circumvent existing security measures). For this reason, ensuring data security at financial institutions requires continuous monitoring, testing, and reinforcement.

The literature and practices surrounding data security and privacy are vast. Different financial organizations may confront different security issues and approach privacy in various ways. Furthermore, planning for security and privacy frequently involves a large number of individuals, including managers, chief information officers, chief security officers, chief technology officers, security experts, network experts, infrastructure engineers, software engineers, data engineers, data architects, and data analysts. This book will concentrate on four major issues of interest to financial data engineers: data privacy regulations, data anonymization, data encryption, and access control.

Data Privacy

Data privacy is a data governance practice that ensures data is collected, stored, processed, and shared in accordance with data protection laws and regulations, as well as the data subject's general interests. Data privacy guarantees that sensitive information is not used for reasons other than those consented to by the data subject or established by law. *Personally identifiable information* (PII) is possibly the most sensitive sort of information. PII refers to any data that can be used either alone or in combination with other data to identify an individual. This can include direct identifiers such as a person's name, address, social security number, or email address, as well as indirect identifiers like birth date, phone number, IP address, or biometric data. In the context of financial markets, financial PII may refer to bank account details, credit card numbers, investment account information, and any other identifiers that could potentially reveal an individual's financial identity.

A number of data regulations and laws have been devised and put into effect internationally in recent years. The most prominent and comprehensive framework is the European Union's *General Data Protection Regulation* (GDPR).[22] In a nutshell, GDPR's main goal is to give EU citizens more control over their personal data. It applies to all EU citizens as well as the entities that do business with them, including those not based in EU countries. GDPR distinguishes between two types of data processing entities: a *data controller* and a *data processor*. A data controller is an entity that collects personal data and determines the purposes (why) for which and the means (how) by which personal data is processed. If multiple data controllers are involved, then the entity is called a *joint controller*. On the other hand, a data processor is an entity that processes data on behalf of the controller. In most cases, the data processor is an external third-party entity (e.g., a payroll company).

Individual rights defined in GDPR relate mostly to the collection, usage, sharing, transfer, and deletion of personal data. Such rights can be grouped into three categories that you are likely to encounter:

Right to access

EU individuals have the right to access and request a copy of their personal data, as well as clarifications on how their data is processed, stored, and used.

Right to be forgotten

EU individuals have the right to request the deletion of their personal data or reject having their data processed.

Data portability

> When feasible, EU individuals should have the right to have their personal data transmitted from one data controller to another.

A variety of similar data protection laws have been introduced worldwide, such as:

- The California Consumer Privacy Act (CCPA) in the US
- The Gramm–Leach–Bliley Act
- Canada's Personal Information Protection and Electronic Documents Act (PIPEDA)
- Japan's Act on Protection of Personal Information (APPI)
- Brazil's General Data Protection Law

With the adoption of these regulations, the demand for privacy-preserving features in system design has increased considerably. As a financial data engineer, your responsibility within this context is related to data collection, visibility, and utilization. For example, if a user consents to the usage of their data for marketing purposes only, then such a restriction needs to be taken into account in the data pipeline(s) that process customer data. A good approach to enforcing data privacy is through data contracts, where all privacy-related requirements are established and agreed upon by both data producers and consumers.

When integrating data privacy elements into system design, a tradeoff might emerge between data confidentiality and data sharing. While limiting data sharing might increase data security and confidentiality, it can also limit prospects for meaningful innovation, both within the financial institution and with external partners. On the other side, enabling excessive data sharing exposes the company to security breaches, legal penalties, and reputational risks.

Ensuring privacy requires having a culture of privacy in the first place. Explaining the impact that privacy infringement can have on the employees is an essential first step. This encourages a due diligence mindset when handling data. Additionally, having management support and interest in enforcing data privacy plays a crucial role. Depending on these and other factors, various financial institutions may invest differently in data privacy standards.[23]

On the methodological side, a variety of data privacy techniques exist. The most effective of such techniques is data anonymization, which I explain in detail in the next section.

Data Anonymization

Data anonymization is a data governance practice that ensures both data security and privacy via transformations that obscure the identifiability of the data. If data is properly anonymized, then it loses its essential identification elements and cannot be linked to specific data objects. This in turn makes it useless if it falls into the wrong hands. In financial institutions, anonymization can be adopted as a good data practice, but it may also be mandated by law. For example, GDPR states that for data to be exempt from certain GRPD privacy restrictions, it needs to be anonymized.[24]

Anonymization strategy

Interestingly, if you check any reference about data anonymization, you will notice that there exists a large discrepancy in the presentation and categorization of data anonymization techniques. Consequently, to establish a baseline, I will initially outline the key factors to consider when devising or choosing a data anonymization approach.

The first element of an anonymization strategy is the *identifiability spectrum*. At one end of the identifiability spectrum, data is completely identifiable. One way to achieve full identifiability is via *direct identifiers*, which refer to values in a dataset that can directly identify a data object without additional information. Examples of direct identifiers could be client name, social security number, financial security identifier, company name, and credit card number. For example, if you know that the ISIN of a company is US5949181045, then you can easily find out that this is Microsoft Corporation. On the other hand, *indirect identifiers or quasi-identifiers* are values that, when combined with other variables in the data, can identify a data object. Examples of indirect identifiers include company domiciliation, market capitalization, price, and the name of the CEO. If your data has information about a company whose CEO in 2023 is Satya Nadella, then it is very likely that we are talking about Microsoft Corporation. Following this logic, we can place direct identifiers on one extreme of the spectrum followed by indirect identifiers. As you obscure or remove data identifiers, you anonymize the data more and more, and it becomes more difficult to identify data objects. At the other end of the spectrum, data is completely anonymized and it is not possible to distinguish one data object from another. To decide where to be on the spectrum of identifiability, you need to consider the risks and costs of reidentification. The higher such risks and costs, the higher the threshold is on the spectrum.[25]

The second element of a data anonymization strategy is *analytical integrity*. Suppose that a financial institution agrees to share some of its internal data with a group of external researchers working on a specific project. To this end, the institution decides to anonymize the data. In this case, the anonymization strategy should take into account the fact that data should still be valid for analysis. For example, if the dataset encompasses certain correlations between the variables that cannot be randomly altered, then it is important to use an anonymization technique that preserves such features in the data.

Reversibility is the third element, which denotes the possibility of reversing the anonymization process by reidentifying the data. If the anonymization is done for data-sharing purposes, then it may not be necessary to reverse the process as it concerns a copy of the data intended for external use. However, if anonymization is done for internal purposes, e.g., credit card numbers, then it may be necessary to introduce reversibility into the anonymization strategy.

The fourth element is *simplicity*. A large number of anonymization techniques are available and they differ in their implementation complexity and interpretability. Simple methods are easy to implement and reverse, while complex techniques require more time and effort.

Fifth, anonymization can be performed statically or dynamically. In static anonymization, data is anonymized and then stored in the final destination for safe future use. Dynamic anonymization, also called interactive anonymization, applies anonymization on the fly to the result of a query or request and not to the entire dataset. When dynamic anonymization is used, an important variable that needs to be taken into account is performance and the speed at which data gets anonymized.

Finally, anonymization can be applied in a deterministic or nondeterministic way. In deterministic anonymization, the outcome of anonymization is always the same even if repeated multiple times. For example, if the name John Smith gets replaced by XXYREQ12, then repeating the process again would replace John Smith with XXYREQ12. In a nondeterministic anonymization, this is not a requirement. For example, the name John Smith can be replaced by a randomly generated string that can change every time you implement the anonymization.

TIP

Data anonymization is an investment that requires effort, expertise, and integration into your financial data infrastructure. For efficiency reasons, I recommend that you first classify your data based on its sensitivity/confidentiality (e.g., Class A is public data, Class B is for internal use, Class G is strictly confidential, etc.) and then anonymize just the most critical data classes.

Anonymization techniques

After outlining your anonymization needs, you can employ a range of techniques for implementation. But before moving forward, it's important to differentiate between an anonymization technique and a measure of its effectiveness.

An anonymization technique takes a raw dataset as input and returns an anonymized dataset as output. An anonymization effectiveness measure evaluates the level of anonymity of an anonymized dataset. Examples of anonymization effectiveness techniques include *k-anonymity*, *l-diversity*, and *t-closeness*. To illustrate the idea, *k*-anonymity, for example, checks whether the information for each data object contained in the dataset cannot be distinguished from at least $k - 1$ data objects whose information also appears in the dataset.[26]

The rest of this section will focus on five common anonymization techniques: generalization, suppression, distortion, swapping, and masking. To illustrate each of these, let's take an initial data sample (Table 5-1) and see how each technique applies to it.

TIP

You can always implement your own anonymization technique. However, before crafting your own solution, you should first consider the available solutions and their use cases. Data anonymization is a major field of study, and you are very likely going to find what you need in the literature.

Table 5-1. Original data before anonymization

ID	Company name	CEO	Headquarters	Revenues	Market capitalization
XYA12F	Standard Steel Corporation	John Smith	New York	$45 mln	$400 mln
BFG76D	Northwest Bank	Lesly Charles	Las Vegas	$5.5 bln	$50 bln
M47GK	General Bicycles Corporation	Mary Jackson	Chicago	$650 mln	$10 bln

Using the generalization technique involves substituting values with less specific yet consistent alternatives. For example, instead of indicating the exact numbers for revenues and market capitalization, we can use ranges. Table 5-2 illustrates the outcome of this generalization strategy.

Table 5-2. Anonymized data after generalization

ID	Company name	CEO	Headquarters	Revenues	Market capitalization
XYA12F	Standard Steel Corporation	John Smith	New York	$0–100 mln	$0–1 bln
BFG76D	Northwest Bank	Lesly Charles	Las Vegas	$5–10 bln	$0–100 bln
M47GK	General Bicycles Corporation	Mary Jackson	Chicago	$0–1 bln	$0–50 bln

Another technique is *suppression*, which simply removes or drops an entire field from a dataset. For example, in our data sample, we may want to suppress the direct identifiers ID and company name by replacing all values with ********. Table 5-3 illustrates the outcome of suppression.

Table 5-3. Anonymized data after suppression

ID	Company name	CEO	Headquarters	Revenues	Market capitalization
********	********	John Smith	New York	$45 mln	$400 mln
********	********	Lesly Charles	Las Vegas	$5.5 bln	$50 bln
********	********	Mary Jackson	Chicago	$650 mln	$10 bln

Another effective technique is *distortion*, which applies mostly to numerical fields and works by adding a certain noise to the values to alter their true value. For example, one can simply generate a random number from a given probability distribution and add it to the value of each record in the column. A large number of formulas can be used for distortion. For illustrative purposes, let's assume that we want to alter the values for revenues by multiplying each number by 1.1, and market capitalization by 1.3. The outcome of this anonymization process is shown in Table 5-4.

Table 5-4. Anonymized data after distortion

ID	Company name	CEO	Headquarters	Revenues	Market capitalization
XYA12F	Standard Steel Corporation	John Smith	New York	$49.5 mln	$520 mln
BFG76D	Northwest Bank	Lesly Charles	Las Vegas	$6.05 bln	$65 bln
M47GK	General Bicycles Corporation	Mary Jackson	Chicago	$715 mln	$13 bln

The next technique is swapping, which works by shuffling the data within one or more fields. For example, in our original data, we could shuffle the company and CEO names as illustrated in Table 5-5.

Table 5-5. Anonymized data after swapping

ID	Company name	CEO	Headquarters	Revenues	Market capitalization
XYA12F	Northwest Bank	John Smith	New York	$45 mln	$400 mln
BFG76D	General Bicycles Corporation	Lesly Charles	Las Vegas	$5.5 bln	$50 bln
M47GK	Standard Steel Corporation	Mary Jackson	Chicago	$650 mln	$10 bln

One of the more popular techniques is masking, which obfuscates sensitive data by using a modified version with modified characters. For example, the ID field in our data sample can be masked by keeping the first character and replacing numbers with 0 and alphabetic characters with 1, as shown in Table 5-6.

Table 5-6. Anonymized data after masking

ID	Company name	CEO	Headquarters	Revenues	Market capitalization
X11001	Standard Steel Corporation	John Smith	New York	$45 mln	$400 mln
B11001	Northwest Bank	Lesly Charles	Las Vegas	$5.5 bln	$50 bln
M0011	General Bicycles Corporation	Mary Jackson	Chicago	$650 mln	$10 bln

In addition to these basic techniques, a variety of more advanced options are available. One prominent example is *differential privacy*, a mathematically rigorous technique that has proven to be quite reliable.[27] Another special technique that has found applications in finance, and in particular financial machine learning, is synthetic data. Synthetic data is machine-generated data that mirrors the properties of original sensitive data. For

example, if we are able to infer the probability distribution of a sensitive dataset (e.g., user account balance), then we can use such probability distribution to generate a synthetic sample that preserves the same statistical properties of the original dataset.

PAYMENT TOKENIZATION

One of the most important applications of data anonymization in finance is payment tokenization. This is a security technique that uses cryptographic algorithms to convert sensitive payment information such as credit card and bank account data into a unique, random string of characters called a "token." Successively, when conducting payment transactions, the token is used instead of the real payment details. If an unauthorized party gets their hands on a token, they won't be able to do anything with it.

Within the payment industry, several participants provide tokenization services, including payment processors and third-party tokenization vendors. Some payment services providers, such as Stripe, provide tokenization-enabled payment hardware or software as part of their service. Once payment tokens are generated, they are stored and secured in a secure vault managed by the tokenization service provider.

With payment tokenization, a business only needs to store the tokens of their customers. When processing client transactions, the business can send the token to the tokenization service provider, which in turn maps the token to the original payment data securely. This technique can be quite useful for businesses that process recurring transactions such as subscriptions or store customer profile details.

Various methods can be employed to generate tokens in payment tokenization. The simplest approach is Random Number Generation (RNG), where tokens are generated using a random number generator to produce a string of numbers or alphanumeric characters. For added security, mathematical algorithms such as hashing or encryption can be employed. A technique called Format-Preserving Encryption (FPE) can be used to encrypt the card number in such a way that the resulting token retains the format of the original card number (e.g., same length and structure).

WARNING

Don't take it for granted that anonymization is bulletproof. Always keep in mind that reidentification risks are present and can change and evolve as a result of multiple factors. For example, in 2006, Netflix launched a one-million-dollar open competition to enhance its movie recommender engine. To this end, Netflix publicly disclosed one hundred million records exposing hundreds of thousands of user ratings from 1999 to 2005. Although the released dataset contained no direct identifiers, two researchers were able to reidentify a subset of people in the data by cross-referencing Netflix data with *IMDB.com* ratings.

Data Encryption

Data encryption is a fundamental practice in information security and privacy. It involves converting data into an unreadable format, rendering it meaningless to unauthorized individuals. Essentially, encryption transforms plain-text (unencrypted) data into ciphertext (encrypted) using a

cryptographic algorithm. This process utilizes an encryption key to encode the data and a decryption key to decode it back to plain text. This way, if encrypted data falls into the wrong hands, then without the decryption key, they will simply see gibberish text. However, this assumes that the decryption key is kept safe!

Data can be encrypted in different states: at rest, in transit, and in use. Data at rest refers to data residing in a storage location such as a hard disk or cloud storage. Data in transit refers to data that is being transferred from one location to another over a network. Data in use is any data that is being processed or data that is temporarily held in memory.

The field of cryptography is quite vast, and a full discussion of encryption methods and techniques is beyond the scope of this book.[28] Nevertheless, to ensure the security of a financial data infrastructure, financial data engineers will benefit from having a basic understanding of the essential concepts and principles of data encryption.

An important distinction to understand is between *symmetric* and *asymmetric* encryption. In symmetric encryption, only one (symmetric) key is used to encrypt and decrypt the data. Symmetric keys are often considered more efficient to generate and faster at data encryption/decryption. However, they need to be shared and stored carefully. This might occasionally necessitate encrypting the key itself using a different encryption key, which could result in a cycle of dependency. The most popular symmetric encryption method is the Advanced Encryption Standard (AES), developed by the US National Institute of Standards and Technology (NIST). Notably, companies like Google utilize the AES to encrypt their data at the storage level.

On the other hand, *asymmetric encryption* uses a pair of keys, called public and private, to encrypt and decrypt the data. Anyone with the public key can encrypt and send data; however, only the private key can decrypt the data. Due to this double-key feature, asymmetric encryption is often considered more secure. However, asymmetric encryption could be more computationally expensive, especially for large data packets, as it relies on large encryption keys.

The most popular asymmetric encryption technique is *Rivest–Shamir–Adleman* (RSA). RSA generates the keys via a factorization operation of two prime numbers. The public RSA can be used to encrypt the data, but only the person who knows the prime numbers can decrypt the data. RSA keys can be very large (e.g., 2,048 or 4,096 bits are typical sizes), and thus their usage might have an impact on performance.

Nowadays, the use of data encryption has become a default and recommended practice both for data security and compliance purposes. This goes without saying for the financial sector. To give an example, let's take the well-known standard, ISO 9564—Personal Identification Number (PIN) management and security. ISO 9564 specifies principles and requirements for reliable and secure management of cardholder Personal Identification Numbers (PINs). PIN codes are used in many places such as automated teller machine (ATM) systems, point-of-sale (POS) terminals, and vending machines. When inserting the PIN, it needs to be transmitted to the issuer for verification. To secure PIN transmission, ISO 9564 requires that the PIN be encrypted, and specifies a list of approved encryp-

tion algorithms: Triple Data Encryption Algorithm (TDEA), RSA, and Advanced Encryption Standard (AES).

Access Control

Access control is a data security practice that allows firms to manage access to their data and related resources. A secure access control policy defines who has access to what, what type of access privileges are assigned, and a disaster management strategy to deal with access anomalies or incidents.

The two main components of access control are *authentication* and *authorization*. Authentication is an identity verification process that checks who is making the access request. Once a user has been authenticated, authorization verifies which resources the user has access to and what access privileges they have.

Access control management is a continuous and primary function within any financial institution. The typical approach to access control involves creating and enforcing a set of guidelines and protocols to be followed when granting access and privileges, as well as a monitoring system to alert against unauthorized access or excessive rights. Although such procedures might differ from one institution to another, a number of best practices have emerged over the years. For example, a quite effective principle is that of *least privileges*, which states that a user or application should be granted the minimal amount of permissions required to perform their tasks. Excess privileges can be quite dangerous and lead to consequential incidents, especially if the user is not aware of them. For example, if a new user is given access and read/write/update/delete privileges to the production database, then data is exposed to deletion or corruption risk.

Another best practice is *multifactor authentication*, which requires going through separate factors to log in—for example, logging in via email plus entering a code that is sent to a linked mobile device. Of similar importance is the practice of access control audit logs, which involves monitoring and collecting information on user activities to detect anomalous or unexpected privileges.

CASE STUDY: PAYMENT CARD INDUSTRY DATA SECURITY STANDARD

To give an empirical overview of how the financial industry formulates data security requirements and policies, let's illustrate the widely used Payment Card Industry Data Security Standard, or for short, PCI DSS.

PCI DSS is a set of policies and procedures intended to ensure the security of payment card transactions and associated data. PCI DSS is defined by the PCI Security Standards Council (SSC). Compliance with PCI DSS is not mandatory by law or regulation. However, it is highly recommended for any institution that stores, processes, and transmits cardholder data. In some cases, such institutions might be required to comply with PCI DSS due to a contractual clause. Furthermore, businesses that demonstrate compliance with PCI DSS are more likely to be trusted in the market.

Cardholder data can be identification data such as PAN, cardholder name, expiry date, and service code, as well as authentication data such as the magnetic stripe data, card verification code (CVC), and PIN code.

To comply with PCI DSS, 12 requirements have been established, which can be grouped into 6 major goals:

- Build and maintain secure networks for conducting card transactions. The recommended approach suggests installing reliable firewalls to block and prevent unauthorized access to the network.
- Protect cardholder data. The standard recommends storing only what's necessary for business operations. Card authentication data should never be stored. When transmitting card data over a network, it should be encrypted.
- Maintain a vulnerability management program to protect against attacks, and perform regular updates and patches of antivirus and operating systems.
- Implement strong access measures via access policies, authentication, and authorization, and ensure the physical security of data.
- Regularly monitor and test networks by tracking all access to network resources and cardholder data and testing security systems.
- Maintain an information security policy that highlights the duties and responsibilities of the personnel and the potential consequences of noncompliance.

For more details on the standard specifications, consult the PCI DSS Quick Reference Guide available on the official PCI DSS web page.

Ongoing efforts are continuously improving security within financial markets. For instance, the European Union is planning to introduce the Digital Operational Resilience Act (DORA) to enhance the digital operational resilience of financial institutions. DORA sets requirements for ICT risk management, incident reporting, and testing to ensure firms can withstand, respond to, and recover from all types of ICT-related disruptions.

Summary

This chapter provided an overview of financial data governance, which we can summarize as follows:

- Defining financial data governance and illustrating its critical importance within the financial domain
- Introduction to data quality, with a discussion of nine dimensions relevant to financial data
- Examination of data integrity through the lens of nine fundamental principles pertinent to financial data
- Illustration of primary security and privacy challenges and best practices affecting most financial institutions

Financial data governance can apply to any aspect of your institutions' data infrastructure, strategy, and business operations. As you read further in this book, you will observe how the different practices and principles discussed in this chapter are employed.

Over the next five chapters, we will go through the financial data engineering lifecycle, where you will learn about the different layers you will need to implement when designing a financial data infrastructure.

[1] For an overview of data quality frameworks, see Corinna Cichy and Stefan Rass' "An Overview of Data Quality Frameworks", *IEEE Access* 7 (2019): 24634–24648.

[2] This approach of defining data quality attributes based on the needs of data consumers, rather than on theoretical findings, is brilliantly illustrated in the work of Richard Y. Wang and Diane M. Strong in their paper "Beyond Accuracy: What Data Quality Means to Data Consumers", *Journal of Management Information Systems* 12, no. 4, (1996): 5–33.

[3] For a good read on this, see Lukas Budach, Moritz Feuerpfeil, Nina Ihde, Andrea Nathansen, Nele Noack, Hendrik Patzlaff, Felix Naumann, and Hazar Harmouch, "The Effects of Data Quality on Machine Learning Performance", *arXiv* preprint arXiv:2207.14529 (July 2022).

[4] If you want to learn more about this issue, I recommend Ramazan Gençay, Michel Dacorogna, Ulrich A. Muller, Olivier Pictet, and Richard Olsen's *An Introduction to High-Frequency Finance* (Elsevier, 2001).

[5] For a good introduction to this topic, I recommend reading Chapter 9 of Pang-Ning Tan, Michael Steinbach, Vipin Kumar, and Anuj Karpatne's *Introduction to Data Mining*, 2nd ed. (Pearson Education, 2019).

[6] For more on this topic, see Carson Kai-Sang Leung, Ruppa K. Thulasiram, and Dmitri A. Bondarenko's "An Efficient System for Detecting Outliers from Financial Time Series", in *Flexible and Efficient Information Handling: Proceedings of the 23rd British National Conference on Databases, BNCOD '06*, Belfast, Northern Ireland, UK, July 18–20, 2006. (Springer Berlin Heidelberg, 2006): 190–198.

[7] For more on this topic, see Kangbok Lee, Yeasung Jeong, Sunghoon Joo, Yeo Song Yoon, Sumin Han, and Hyeoncheol Baik's "Outliers in Financial Time Series Data: Outliers, Margin Debt, and Economic Recession", *Machine Learning with Applications* 10 (December 2022): 100420.

[8] For a good read on this topic, see Christian T. Brownlees and Giampiero M. Gallo's "Financial Econometric Analysis at Ultra-High Frequency: Data Handling Concerns", *Computational Statistics & Data Analysis* 51, no. 4 (December 2006): 2232–2245.

[9] For more on this topic, I highly recommend John Adams, Darren Hayunga, Sattar Mansi, David Reeb, and Vincenzo Verardi's "Identifying and Treating Outliers in Finance", *Financial Management* 48, no. 2 (March 2019): 345–384.

10 For more on this topic, see Sagar P. Kothari, Jay Shanken, and Richard G. Sloan's "Another Look at the Cross-Section of Expected Stock Returns", *The Journal of Finance* 50, no. 1 (March 1995): 185–224.

11 For a good read, start with Ninareh Mehrabi, Fred Morstatter, Nripsuta Saxena, Kristina Lerman, and Aram Galstyan, "A Survey on Bias and Fairness in Machine Learning", *ACM Computing Surveys (CSUR)* 54, no. 6 (July 2021): 1–35.

12 For more on the Lipper database, see Mila Getmansky, Andrew W. Lo, and Shauna X. Mei's chapter, "Sifting Through the Wreckage: Lessons from Recent Hedge-Fund Liquidations", in *The World of Hedge Funds: Characteristics and Analysis*, H Gifford Fong, ed. (World Scientific Publishing Company, 2005): 7–47.

13 To test the code quickly online, you can use OneCompiler.

14 For a good read on this topic, see Pat Langley's "Selection of Relevant Features in Machine Learning", in *Proceedings of the AAAI Fall Symposium on Relevance (1994),* vol. 184 (AAAI Press, 1994): 245–271.

15 To learn more about trading strategies and their data requirements, I recommend Eugene A. Durenard's *Professional Automated Trading: Theory and Practice* (Wiley, 2013).

16 For an excellent treatment of data catalogs, I recommend Ole Olesen-Bagneux's *The Enterprise Data Catalog* (O'Reilly, 2023).

17 For more on this topic, see Ali M. Al-Khouri's "Data Ownership: Who Owns "My Data?", *International Journal of Management & Information Technology* 2, no. 1 (November 2012): 1-8.

18 For a detailed study on this topic, I recommend Marshall Van Alstyne, Erik Brynjolfsson, and Stuart Madnick's "Why Not One Big Database? Principles for Data Ownership", *Decision Support Systems* 15, no. 4 (December 1995): 267-284.

19 For a good read on data contracts, I highly recommend the book by Andrew Jones, *Driving Data Quality with Data Contracts: A Comprehensive Guide to Building Reliable, Trusted, and Effective Data Platforms* (Packt, 2023).

20 To read more about portfolio reconciliation and the technological side of it, I recommend the ISDA's paper, "Portfolio Reconciliation in Practice".

21 For more on payment reconciliation, see "Payment Reconciliation 101: How It Works and Best Practices for Businesses", by Stripe.

22 For a practical introduction to GDPR, I strongly recommend Paul Voigt and Axel von dem Bussche's *The ER General Data Protection Regulation (GDPR): A Practical Guide*, 1st ed. (Springer, 2017).

23 For a comparative study on this topic, I recommend Lorrie Faith Cranor, Kelly Idouchi, Pedro Giovanni Leon, Manya Sleeper, and Blase Ur's "Are They Actually Any Different? Comparing Thousands of Financial Institutions' Privacy Practices", presented at the 12th Annual Workshop on the Economics of Information Security (WEIS 2013).

24 A key distinction to remember under GDPR is the difference between anonymization and pseudonymization. According to GDPR, anonymization refers to the process of irreversibly removing personal identifiers from data so that individuals can no longer be identified, making the data completely anonymous and not subject to GDPR. Pseudonymization, on the other hand, involves processing data in such a way that individuals cannot be identified without additional information, which is kept separately and protected. Unlike anonymized data, pseudonymized data is still considered personal data under GDPR and remains

subject to its regulations, as the potential to reidentify individuals exists if the additional information is accessed.

25 For an introduction to risk-based data anonymization techniques, I highly recommend the book by Khaled El Emam and Luk Arbuckle, *Anonymizing Health Data: Case Studies and Methods to Get You Started* (O'Reilly, 2013).

26 To learn more about these measures, I highly recommend the publication by Ninghui Li, Tiancheng Li, and Suresh Venkatasubramanian, "t-Closeness: Privacy Beyond k-Anonymity and l-Diversity", in the *2007 IEEE 23rd International Conference on Data Engineering* (IEEE, 2006): 106-115.

27 See for example Cynthia Dwork's "Differential Privacy: A Survey of Results", in the *Proceedings of the 5th International Conference on the Theory and Applications of Models of Computation* (Springer, 2008): 1–19.

28 Interested readers are encouraged to read Jonathan Katz and Yehuda Lindell's *Introduction to Modern Cryptography: Principles and Protocols* (Chapman and Hall/CRC, 2021).